

Coupon Purchase Prediction

```
In [5]: import warnings
import numpy as np
import pandas as pd
import sklearn as sk
import matplotlib
import seaborn as sns
import matplotlib.pyplot as plt
import collections
from collections import Counter
from sklearn.decomposition import PCA
%matplotlib inline
warnings.filterwarnings('ignore')

import xgboost as xgb
from scipy import stats
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_val_score
from sklearn.metrics import precision_score
from sklearn import preprocessing
from sklearn.preprocessing import PowerTransformer
from sklearn.preprocessing import LabelEncoder
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import classification_report

import os
data_folder = "dataset/"

pdf_files = [f for f in os.listdir(data_folder) if f.endswith('.csv')]

print(f"Found {len(pdf_files)} CSV files.")
```

Found 8 CSV files.

Prepping the Data

```
In [18]: df = dict()
df["hh_demographic"] = pd.read_csv(data_folder + "hh_demographic.csv")
demographic=df["hh_demographic"]
demographic["MARITAL_STATUS_CODE"].replace(['A', 'B', 'U'], ['Married', 'Unknown', 'Si
demographic
```

```
Out[18]:
```

	AGE_DESC	MARITAL_STATUS_CODE	INCOME_DESC	HOMEOWNER_DESC	HH_COMP_DI
0	65+	Married	35-49K	Homeowner	2 Adults No t
1	45-54	Married	50-74K	Homeowner	2 Adults No t
2	25-34	Single	25-34K	Unknown	2 Adults t
3	25-34	Single	75-99K	Homeowner	2 Adults t
4	45-54	Unknown	50-74K	Homeowner	Single Fern
...	...	...	...	...	...
796	35-44	Single	50-74K	Homeowner	2 Adults No t
797	45-54	Married	75-99K	Homeowner	Unknc
798	45-54	Single	35-49K	Unknown	Single M
799	25-34	Single	50-74K	Homeowner	2 Adults No t
800	25-34	Single	Under 15K	Unknown	2 Adults t

801 rows × 8 columns

```
In [17]: df["campaign_desc"] = pd.read_csv(data_folder+"campaign_desc.csv")
campaign_desc=df["campaign_desc"]
#Sort campaign by start date
campaign_desc=campaign_desc.sort_values(by=['START_DAY', 'CAMPAIGN'],ascending=True)
campaign_desc
```

Out[17]:	DESCRIPTION	CAMPAIGN	START_DAY	END_DAY	
	29	TypeA	26	224	264
	28	TypeC	27	237	300
	27	TypeB	28	259	320
	26	TypeB	29	281	334
	25	TypeA	30	323	369
	23	TypeB	1	346	383
	24	TypeB	2	351	383
	20	TypeC	3	356	412
	22	TypeB	4	372	404
	21	TypeB	5	377	411
	19	TypeC	6	393	425
	18	TypeB	7	398	432
	17	TypeA	8	412	460
	16	TypeB	9	435	467
	15	TypeB	10	463	495
	13	TypeB	11	477	523
	14	TypeB	12	477	509
	12	TypeA	13	504	551
	10	TypeC	14	531	596
	1	TypeC	15	547	708
	11	TypeB	16	561	593
	9	TypeB	17	575	607
	7	TypeA	18	587	642
	8	TypeB	19	603	635
	3	TypeC	20	615	685
	5	TypeB	21	624	656
	6	TypeB	22	624	656
	4	TypeB	23	646	684
	0	TypeB	24	659	719
	2	TypeB	25	659	691

```
In [8]: #We exclude the last five campaigns filtering on campaigns starting before days 615  
campaign_desc = campaign_desc[campaign_desc['START_DAY']<615]
```

```
In [9]: df["campaign_table"] = pd.read_csv(data_folder+"campaign_table.csv")  
campaign_table=df["campaign_table"]  
campaign_table.head(10)
```

```
Out[9]:
```

	DESCRIPTION	household_key	CAMPAIGN
0	TypeA	17	26
1	TypeA	27	26
2	TypeA	212	26
3	TypeA	208	26
4	TypeA	192	26
5	TypeA	187	26
6	TypeA	183	26
7	TypeA	142	26
8	TypeA	140	26
9	TypeA	134	26

```
In [16]: #We call campaign the new dataframe merging the dataset  
campaign = pd.merge(campaign_desc[['CAMPAIGN','START_DAY']],campaign_table[['househ  
#Count number of campaign per household  
campaign['#campaign']=campaign.groupby(by='household_key')['CAMPAIGN'].transform('c  
#Delete useless column  
campaign=campaign.drop(columns=['CAMPAIGN','START_DAY'])  
#Delete duplicates  
campaign.drop_duplicates(subset=['household_key', '#campaign'], keep="first", inpla  
campaign
```

```
Out[16]:
```

	household_key	#campaign
0	17	5
1	27	7
2	212	5
3	208	7
4	192	3
...	...	...
6110	1744	1
6115	1594	1
6119	1479	1
6121	1447	1
6134	1223	1

1564 rows × 2 columns

```
In [13]: #Read the coupon_redempt table
df["coupon_redempt"] = pd.read_csv(data_folder+"coupon_redempt.csv")
coupon_redempt=df["coupon_redempt"]
#Keep only coupon redeemed before DAY 615
coupon_redempt=coupon_redempt[coupon_redempt['DAY']<615]
#Drop useless columns
coupon_redempt=coupon_redempt.drop(columns=['DAY','COUPON_UPC'])
#Keep only one occurrence of coupon redeemed by campaign
coupon_redempt.drop_duplicates(subset=['household_key', 'CAMPAIGN'], keep="first",
#Count number of campaign the customer redeemed at least one coupon
redemption_per_household=coupon_redempt.groupby(['household_key'], as_index=False)[
redemption_per_household
```

Out[13]:

	household_key	redeemed
0	1	2
1	8	1
2	13	6
3	14	1
4	18	3
...	...	...
392	2481	2
393	2488	4
394	2489	5
395	2496	3
396	2500	1

397 rows × 2 columns

```
In [19]: #Merging of campaign and coupon redemption tables
temp = pd.merge(campaign, redemption_per_household, on=['household_key'], how="left")
#Creation of our output variable
temp["Sensitivity"] = np.where(temp["redeemed"] > 0, 'Sensible', 'Not sensible')
#Creation of our aggregated dataset. We use the inner join to keep only customers f
dataset = pd.merge(demographic, temp[['household_key', 'Sensitivity']], on=['househol
dataset
```

Out[19]:

	AGE_DESC	MARITAL_STATUS_CODE	INCOME_DESC	HOMEOWNER_DESC	HH_COMP_D
0	65+	Married	35-49K	Homeowner	2 Adults No t
1	45-54	Married	50-74K	Homeowner	2 Adults No t
2	25-34	Single	25-34K	Unknown	2 Adults t
3	25-34	Single	75-99K	Homeowner	2 Adults t
4	45-54	Unknown	50-74K	Homeowner	Single Ferr
...	...	...	...	...	
746	35-44	Single	50-74K	Homeowner	2 Adults No t
747	45-54	Married	75-99K	Homeowner	Unknc
748	45-54	Single	35-49K	Unknown	Single Iv
749	25-34	Single	50-74K	Homeowner	2 Adults No t
750	25-34	Single	Under 15K	Unknown	2 Adults t

751 rows × 9 columns

```
In [21]: # Load the dataset
df["transaction_data"] = pd.read_csv(data_folder+"transaction_data.csv")
transaction=df["transaction_data"]
#Keep transaction before day 615
transaction=transaction[transaction['DAY']<615]
#Exclude transactions related to returns
transaction=transaction[transaction['SALES_VALUE']>0]
transaction=transaction[transaction['QUANTITY']>0]
transaction.head(20)
```

Out[21]:

	household_key	BASKET_ID	DAY	PRODUCT_ID	QUANTITY	SALES_VALUE	STORE_ID
0	2375	26984851472	1	1004906	1	1.39	364
1	2375	26984851472	1	1033142	1	0.82	364
2	2375	26984851472	1	1036325	1	0.99	364
3	2375	26984851472	1	1082185	1	1.21	364
4	2375	26984851472	1	8160430	1	1.50	364
5	2375	26984851516	1	826249	2	1.98	364
6	2375	26984851516	1	1043142	1	1.57	364
7	2375	26984851516	1	1085983	1	2.99	364
8	2375	26984851516	1	1102651	1	1.89	364
9	2375	26984851516	1	6423775	1	2.00	364
10	2375	26984851516	1	9487839	1	2.00	364
11	1364	26984896261	1	842930	1	2.19	31742
12	1364	26984896261	1	897044	1	2.99	31742
13	1364	26984896261	1	920955	1	3.09	31742
14	1364	26984896261	1	937406	1	2.50	31742
15	1364	26984896261	1	981760	1	0.60	31742
16	1130	26984905972	1	833715	2	0.34	31642
17	1130	26984905972	1	866950	2	0.34	31642
18	1130	26984905972	1	1022843	2	0.34	31642
19	1130	26984905972	1	1048462	1	1.19	31642

```
In [22]: #Calculate total sales per customer
total_sales=transaction.groupby(by='household_key', as_index=False)['SALES_VALUE'].
#Calculate total number of visits per customer
total_visits=transaction.groupby(['household_key'], as_index=False)['BASKET_ID'].ag
#Calculate median basket amount per customer
temp_basket=transaction.groupby(['household_key', 'BASKET_ID'], as_index=False)['SA
temp_median_basket=temp_basket.groupby(['household_key'], as_index=False)['SALES_VA
#Calculate average product price bought per customer
temp_product=transaction.groupby(['household_key'], as_index=False)['SALES_VALUE'].
dataset=dataset.merge(total_sales,on='household_key').merge(total_visits,on='househ
dataset=dataset.drop(columns=['household_key'])
dataset
```

Out[22]:

	AGE_DESC	MARITAL_STATUS_CODE	INCOME_DESC	HOMEOWNER_DESC	HH_COMP_DI
0	65+	Married	35-49K	Homeowner	2 Adults No t
1	45-54	Married	50-74K	Homeowner	2 Adults No t
2	25-34	Single	25-34K	Unknown	2 Adults t
3	25-34	Single	75-99K	Homeowner	2 Adults t
4	45-54	Unknown	50-74K	Homeowner	Single Ferr
...	...	...	...	...	
746	35-44	Single	50-74K	Homeowner	2 Adults No t
747	45-54	Married	75-99K	Homeowner	Unknc
748	45-54	Single	35-49K	Unknown	Single Iv
749	25-34	Single	50-74K	Homeowner	2 Adults No t
750	25-34	Single	Under 15K	Unknown	2 Adults t

751 rows × 12 columns

Observing the Data

```
In [23]: print(dataset.shape)
(751, 12)

In [24]: print(dataset.info())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 751 entries, 0 to 750
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   AGE_DESC              751 non-null   object 
 1   MARITAL_STATUS_CODE   751 non-null   object 
 2   INCOME_DESC           751 non-null   object 
 3   HOMEOWNER_DESC        751 non-null   object 
 4   HH_COMP_DESC          751 non-null   object 
 5   HOUSEHOLD_SIZE_DESC   751 non-null   object 
 6   KID_CATEGORY_DESC     751 non-null   object 
 7   Sensitivity           751 non-null   object 
 8   Total_sales           751 non-null   float64
 9   total_visits          751 non-null   int64  
10   median_basket         751 non-null   float64
11   avg_price             751 non-null   float64
dtypes: float64(3), int64(1), object(8)
memory usage: 70.5+ KB
None

```

```

In [25]: pd.options.display.float_format = "{:.2f}".format
dataset.describe()

```

```

Out[25]:

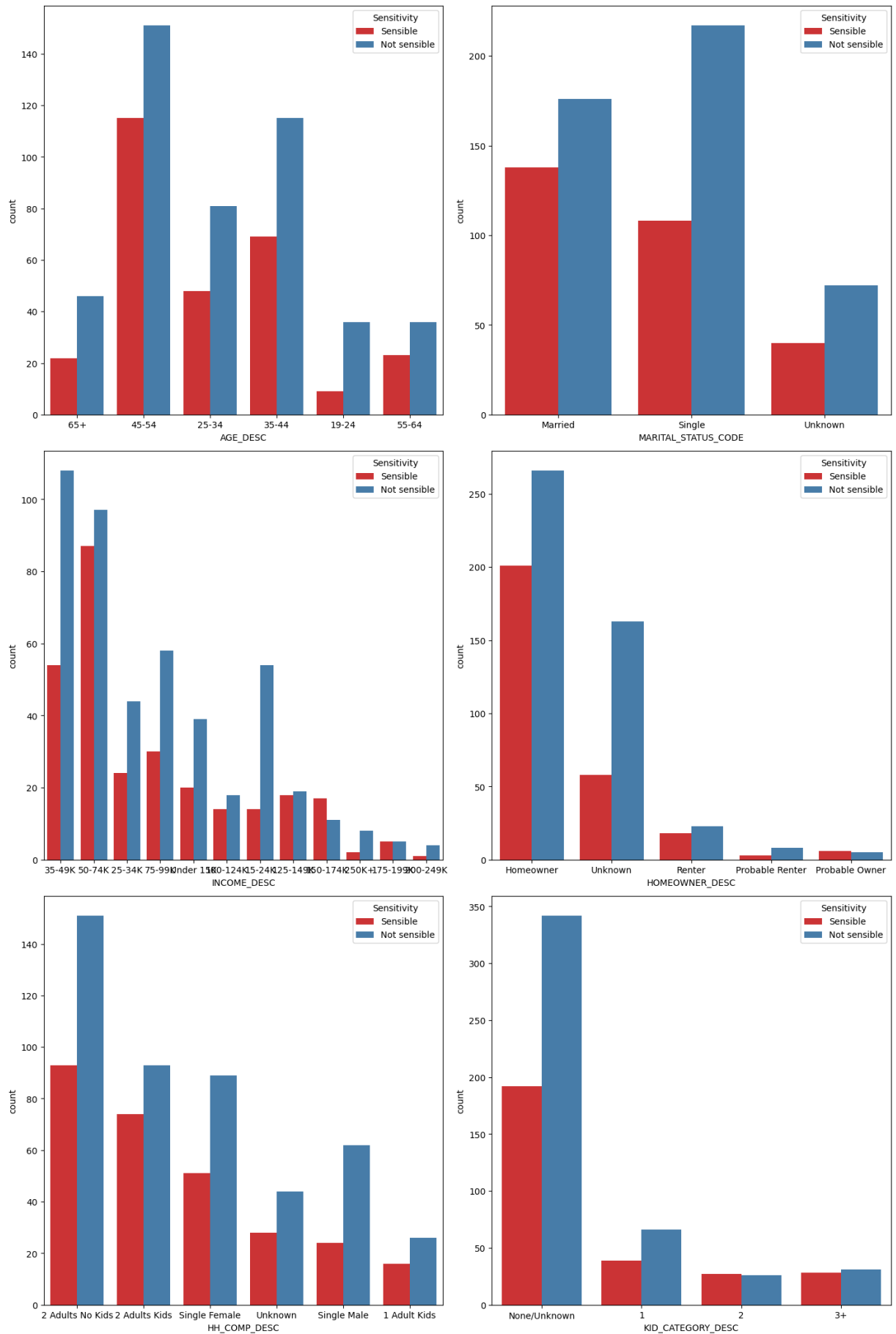
```

	Total_sales	total_visits	median_basket	avg_price
count	751.00	751.00	751.00	751.00
mean	4856.60	152.94	27.84	3.16
std	3164.52	114.18	19.33	0.76
min	603.22	26.00	1.38	1.31
25%	2549.38	86.00	15.00	2.67
50%	4055.48	121.00	23.55	3.08
75%	6135.14	186.00	34.98	3.54
max	23805.83	1161.00	128.88	9.41

```

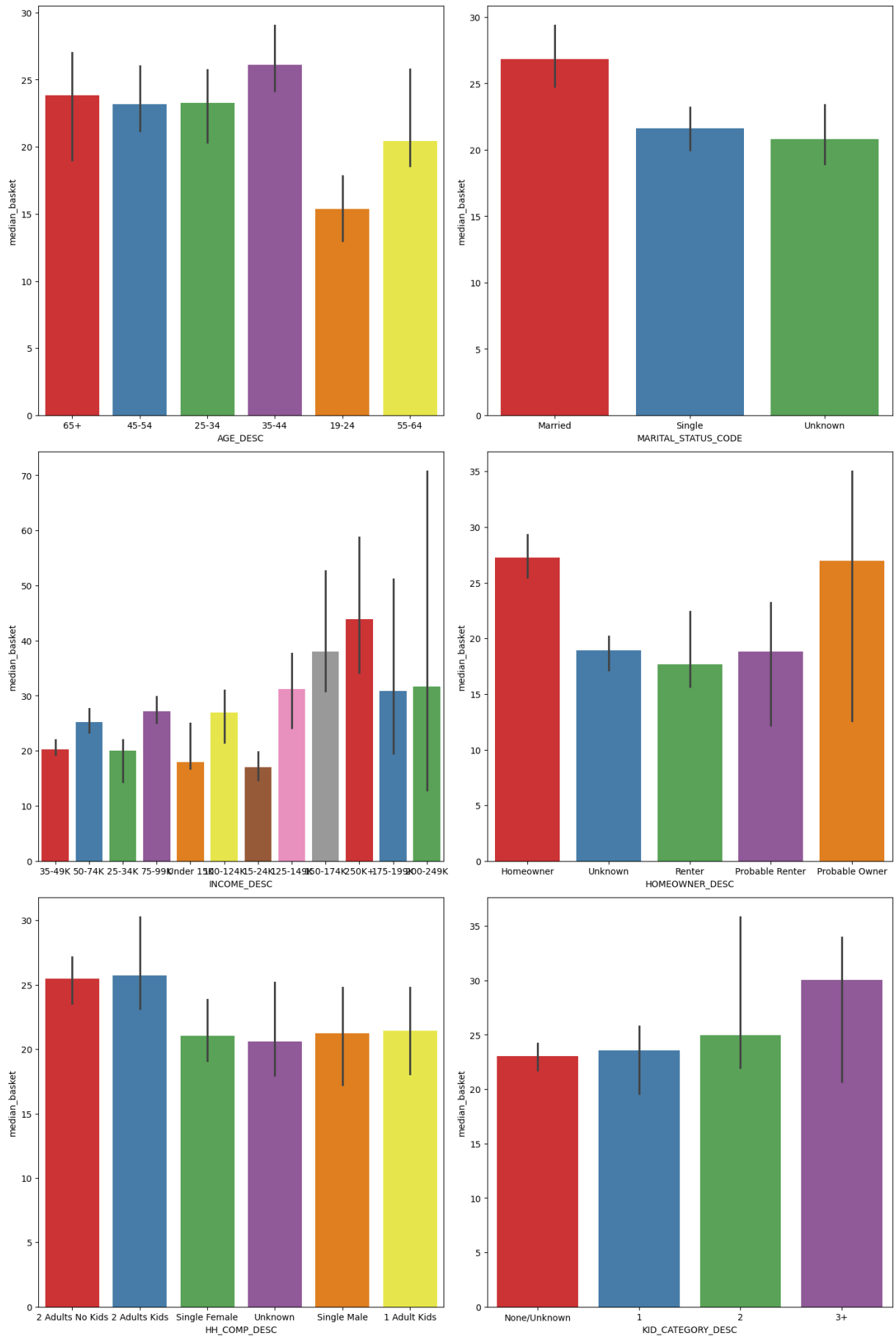
In [26]: categorical_vars = ['AGE_DESC', 'MARITAL_STATUS_CODE', 'INCOME_DESC', 'HOMEOWNER_DESC']
num_plots = len(categorical_vars)
total_cols = 2
total_rows = num_plots//total_cols
fig, axs = plt.subplots(nrows=total_rows, ncols=total_cols,
                        figsize=(7*total_cols, 7*total_rows), constrained_layout=True)
for i, var in enumerate(categorical_vars):
    row = i//total_cols
    pos = i % total_cols
    plot = sns.countplot(x=var, data=dataset, ax=axs[row][pos], hue='Sensitivity', pa

```



```
In [27]: categorical_vars = ['AGE_DESC', 'MARITAL_STATUS_CODE', 'INCOME_DESC', 'HOMEOWNER_DESC', 'HH_COMP_DESC', 'KID_CATEGORY_DESC']
```

```
num_plots = len(categorical_vars)
total_cols = 2
total_rows = num_plots//total_cols
fig, axs = plt.subplots(nrows=total_rows, ncols=total_cols,
                        figsize=(7*total_cols, 7*total_rows), constrained_layout=True)
for i, var in enumerate(categorical_vars):
    row = i//total_cols
    pos = i % total_cols
    plot = sns.barplot(x=var, y='median_basket', data=dataset, ax=axs[row][pos], pale
```



```
In [28]: target = dataset['Sensitivity']
```

```

counter = Counter(target)
for k,v in counter.items():
    per = v / len(target) * 100
    print('Class=%s, Count=%d, Percentage=%.2f%%' % (k, v, per))

```

Class=Sensible, Count=286, Percentage=38.08%

Class=Not sensible, Count=465, Percentage=61.92%

```

In [29]: #1. Split data into X and Y
X=dataset.drop(columns=['Sensitivity'])
Y=dataset['Sensitivity']

#2.A. Encode string class values as integers
label_encoder = preprocessing.LabelEncoder()
label_encoder = label_encoder.fit(dataset['Sensitivity'])
label_encoded_y = label_encoder.transform(dataset['Sensitivity'])

#2.A. Encode Income values as integers
X['INCOME_DESC'].replace(['Under 15K', '15-24K', '25-34K', '35-49K', '50-74K', '75-
#2.A. Encode Income values as integers
X['AGE_DESC'].replace(['19-24', '25-34', '35-44', '45-54', '55-64', '65+'],[0,1,2,3
#2.A. Label encoding the other categorical data
labelencoder_X_1 = LabelEncoder()
X['MARITAL_STATUS_CODE'] = labelencoder_X_1.fit_transform(X['MARITAL_STATUS_CODE'])
labelencoder_X_2 = LabelEncoder()
X['HOMEOWNER_DESC'] = labelencoder_X_2.fit_transform(X['HOMEOWNER_DESC'])
labelencoder_X_3 = LabelEncoder()
X['HH_COMP_DESC'] = labelencoder_X_3.fit_transform(X['HH_COMP_DESC'])
labelencoder_X_4 = LabelEncoder()
X['HOUSEHOLD_SIZE_DESC'] = labelencoder_X_4.fit_transform(X['HOUSEHOLD_SIZE_DESC'])
X["KID_CATEGORY_DESC"].replace(['None/Unknown', '3+'],[0,3],inplace=True)
X['HOUSEHOLD_SIZE_DESC'] = X.HOUSEHOLD_SIZE_DESC.astype(float)
X['KID_CATEGORY_DESC'] = X.KID_CATEGORY_DESC.astype(float)

```

```

In [32]: #Let's plot the Skewness by decreasing order
num_feats=X.dtypes[X.dtypes!='object'].index
skew_feats=X[num_feats].skew().sort_values(ascending=False)
skewness=pd.DataFrame({'Skew':skew_feats})
skewness

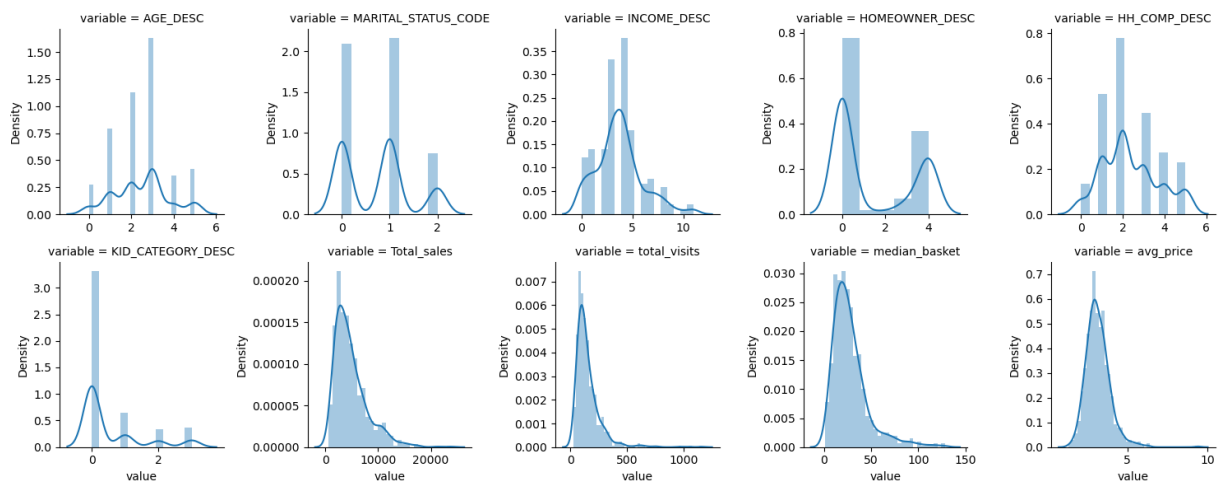
```

Out[32]:

	Skew
total_visits	3.49
median_basket	1.95
KID_CATEGORY_DESC	1.69
Total_sales	1.63
avg_price	1.42
HOUSEHOLD_SIZE_DESC	1.06
INCOME_DESC	0.68
HOMEOWNER_DESC	0.63
MARITAL_STATUS_CODE	0.43
HH_COMP_DESC	0.39
AGE_DESC	0.10

```
In [ ]: df = pd.DataFrame(data=X, columns=['AGE_DESC', 'MARITAL_STATUS_CODE', 'INCOME_DESC', '
nd = pd.melt(df, value_vars =df )
n1 = sns.FacetGrid (nd, col='variable', col_wrap=5, sharex=False, sharey = False)
n1 = n1.map(sns.distplot, 'value')
n1
```

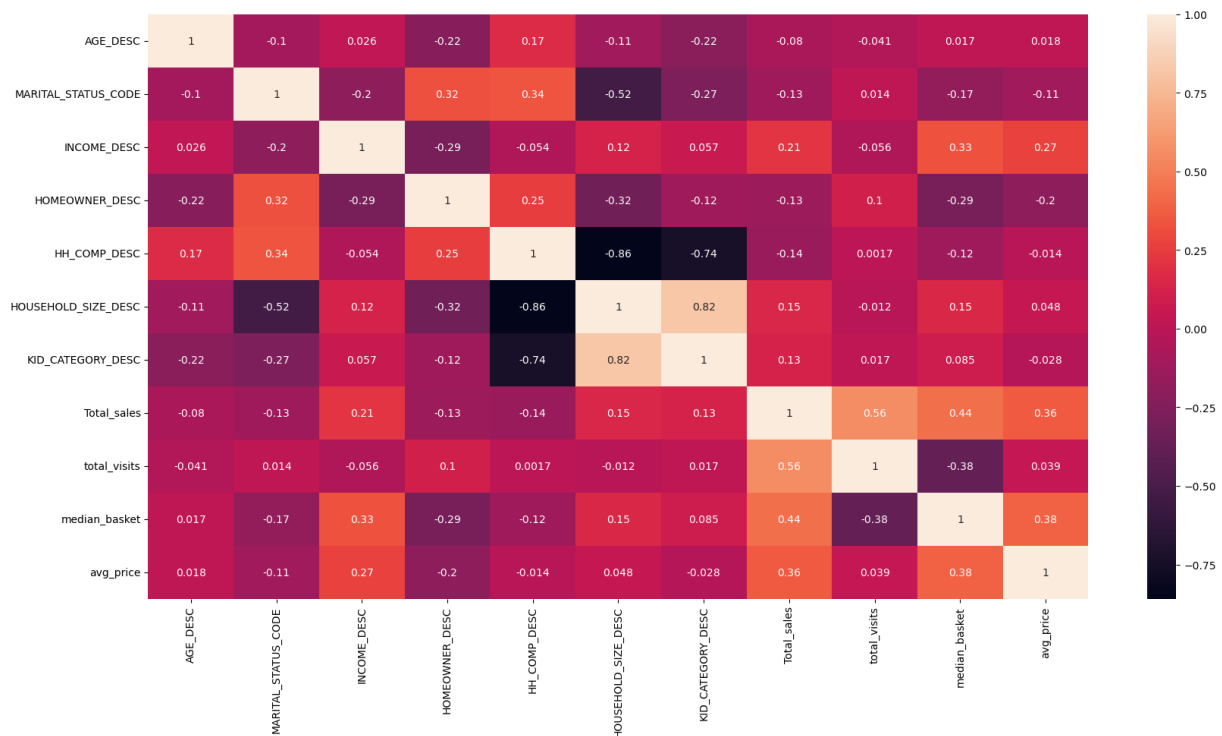
Out[]: <seaborn.axisgrid.FacetGrid at 0x16a80975c40>



```
In [33]: # Finding the relations between the variables
plt.figure(figsize=(20,10))
c= X.corr(method='spearman')
sns.heatmap(c, annot=True)
c
```

Out[33]:

	AGE_DESC	MARITAL_STATUS_CODE	INCOME_DESC	HOMEOWNER_
AGE_DESC	1.00	-0.10	0.03	
MARITAL_STATUS_CODE	-0.10	1.00	-0.20	
INCOME_DESC	0.03	-0.20	1.00	
HOMEOWNER_DESC	-0.22	0.32	-0.29	
HH_COMP_DESC	0.17	0.34	-0.05	
HOUSEHOLD_SIZE_DESC	-0.11	-0.52	0.12	
KID_CATEGORY_DESC	-0.22	-0.27	0.06	
Total_sales	-0.08	-0.13	0.21	
total_visits	-0.04	0.01	-0.06	
median_basket	0.02	-0.17	0.33	
avg_price	0.02	-0.11	0.27	



```
In [34]: #remove attribute
X=X.drop(columns=['HOUSEHOLD_SIZE_DESC'])
```

Prepping Training and Testing Data

```
In [35]: X_train, X_test, y_train, y_test = train_test_split(X, label_encoded_y, test_size=0
```

```
In [36]: #instantiate
pt = PowerTransformer(method='yeo-johnson', standardize=True)
```

```

#Fit the data to the powertransformer
rescaler = pt.fit(X_train)

#Lets get the Lambdas that were found
print (rescaler.lambdas_)

calc_lambdas = rescaler.lambdas_

#Transform the data
X_train_resc = rescaler.transform(X_train)
X_test_resc=rescaler.transform(X_test)

#Pass the transformed data into a new dataframe
df_xt = pd.DataFrame(data=X_train_resc, columns=['AGE_DESC', 'MARITAL_STATUS_CODE', '
df_xt.describe()

```

```

[ 0.80567994 -0.00742693  0.62559009 -0.8781086   0.54468515 -2.93541146
 0.0473906  -0.18151033  0.15899538 -0.38965469]

```

```

Out[36]:

```

	AGE_DESC	MARITAL_STATUS_CODE	INCOME_DESC	HOMEOWNER_DESC	HH_COMP_
count	525.00	525.00	525.00	525.00	5
mean	0.00	-0.00	-0.00	-0.00	
std	1.00	1.00	1.00	1.00	
min	-2.03	-1.13	-1.99	-0.81	
25%	-0.33	-1.13	-0.24	-0.81	
50%	0.43	0.56	0.21	-0.81	
75%	0.43	0.56	0.63	1.29	
max	1.84	1.54	2.74	1.29	

```

In [37]:
pca = PCA().fit(X_train_resc)

plt.rcParams["figure.figsize"] = (12,11)

fig, ax = plt.subplots()
xi = np.arange(1, 11, step=1)
y = np.cumsum(pca.explained_variance_ratio_)

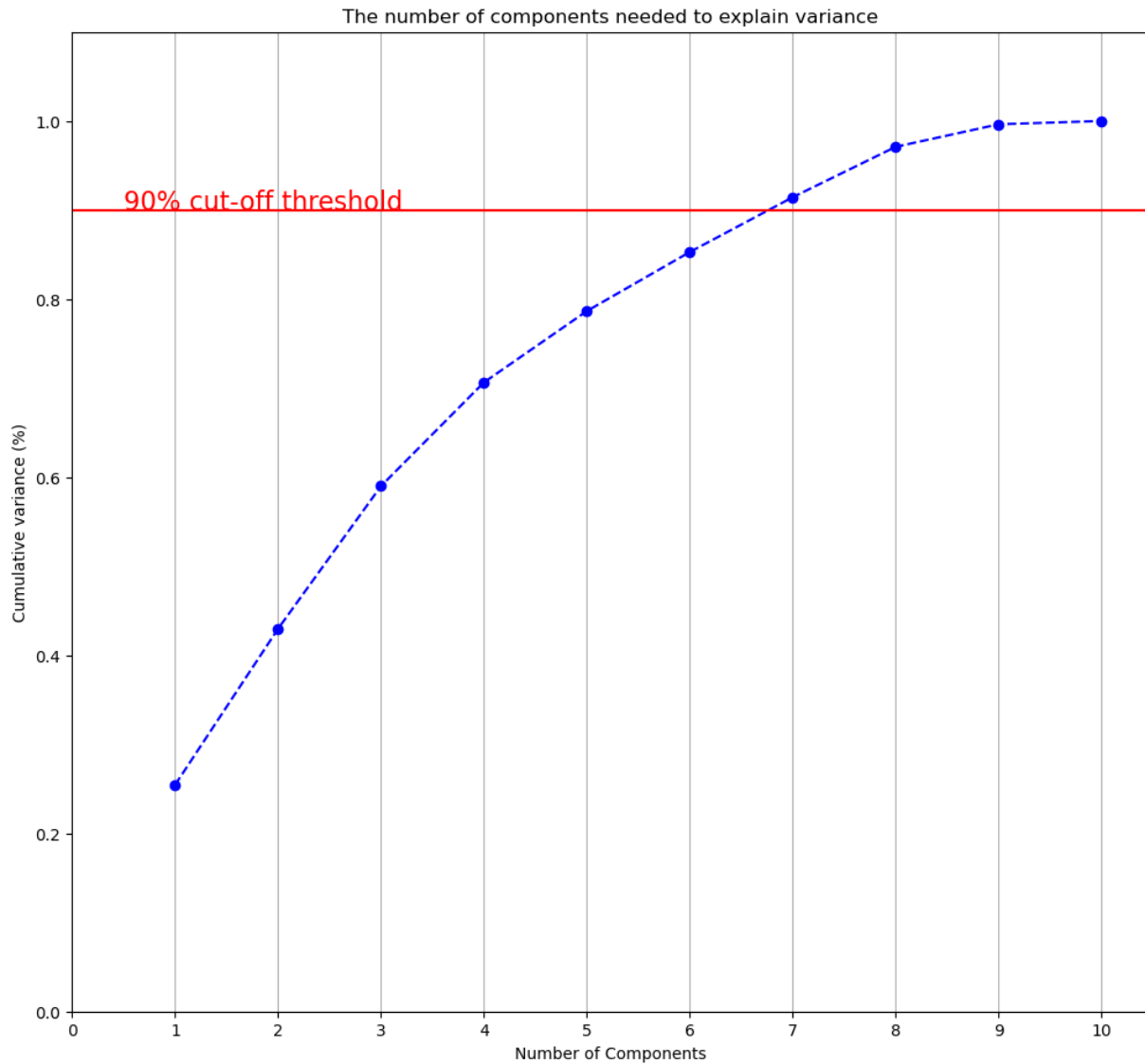
plt.ylim(0.0,1.1)
plt.plot(xi, y, marker='o', linestyle='--', color='b')

plt.xlabel('Number of Components')
plt.xticks(np.arange(0, 11, step=1))
plt.ylabel('Cumulative variance (%)')
plt.title('The number of components needed to explain variance')

plt.axhline(y=0.90, color='r', linestyle='-')
plt.text(0.5, 0.90, '90% cut-off threshold', color = 'red', fontsize=16)

```

```
ax.grid(axis='x')
plt.show()
```



```
In [38]: # on standardized data
pca_std = PCA(n_components=7).fit(X_train_resc)
X_train_PCA = pca_std.transform(X_train_resc)
X_test_PCA = pca_std.transform(X_test_resc)
pca_std.explained_variance_ratio_
```

```
Out[38]: array([0.25450609, 0.17523455, 0.16027727, 0.11641416, 0.08013689,
                0.06630728, 0.06154442])
```

Training the Model

```
In [46]: #check the performance of the XGBoost model without tune parameters
# fit model on training data
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_validate
model = xgb.XGBClassifier()
kfold = StratifiedKFold(n_splits=5, random_state=7, shuffle=True)
```

```

Accuracy = cross_validate(model, X_train_PCA, y_train, cv=kfold,scoring='accuracy')
Precision = cross_validate(model, X_train_PCA, y_train, cv=kfold,scoring='precision')

accuracy_scores = Accuracy['test_score']
precision_scores = Precision['test_score']

print("Accuracy: %.1f%% (%.1f%%)" % (accuracy_scores.mean()*100, accuracy_scores.st
print("Precision: %.1f%% (%.1f%%)" % (precision_scores.mean()*100, precision_scores

```

Accuracy: 66.1% (3.5%)
Precision: 54.6% (6.8%)

```

In [42]: # grid search to tune algorithm
model = xgb.XGBClassifier()
n_estimators = [50,100, 200, 300, 400,450, 500,600,1000]
learning_rate = [0.01,0.05,0.1,0.2]
max_depth= range(2,8)
gamma=[0, 0.25, 0.5, 0.7, 0.9, 1.0]

param_grid = dict(gamma=gamma,learning_rate=learning_rate, n_estimators=n_estimator
eval_set=[(X_train_PCA, y_train), (X_test_PCA, y_test)]
kfold = StratifiedKFold(n_splits=5, random_state=7,shuffle=True)
grid_search = GridSearchCV(model, param_grid,scoring='precision', n_jobs=-1, cv=kfo
grid_result = grid_search.fit(X_train_PCA, y_train,eval_set=eval_set,verbose=20)

# summarize result
print("Best: %.1f%% using %s" % (grid_result.best_score_*100, grid_result.best_para

[0]      validation_0-logloss:0.64734      validation_1-logloss:0.67763
[20]     validation_0-logloss:0.55785      validation_1-logloss:0.63380
[40]     validation_0-logloss:0.50852      validation_1-logloss:0.62398
[20]     validation_0-logloss:0.55785      validation_1-logloss:0.63380
[40]     validation_0-logloss:0.50852      validation_1-logloss:0.62398
[49]     validation_0-logloss:0.49250      validation_1-logloss:0.61926
Best: 62.0% using {'gamma': 1.0, 'learning_rate': 0.1, 'max_depth': 2, 'n_estimators
': 50}

```

```

In [63]: # fit model on training data
model = xgb.XGBClassifier(learning_rate = 0.1,\
                           max_depth=2,\
                           n_estimators=50,\
                           gamma=1.0,\
                           objective = 'binary:logistic',\
                           eval_metric = 'logloss',\
                           )

kfold = StratifiedKFold(n_splits=5, random_state=7,shuffle=True)

Accuracy = cross_validate(model, X_train_PCA, y_train, cv=kfold,scoring='accuracy')
Precision = cross_validate(model, X_train_PCA, y_train, cv=kfold,scoring='precision')

accuracy_scores = Accuracy['test_score']
precision_scores = Precision['test_score']

print("Accuracy: %.1f%%" % (accuracy_scores.mean()*100))
print("Precision: %.1f%%" % (precision_scores.mean()*100))

```

Accuracy: 68.8%
Precision: 62.0%

Testing the Model

```
In [64]: model = xgb.XGBClassifier(learning_rate = 0.1,\
                                   max_depth=2,\
                                   n_estimators=50,\
                                   gamma=1.0,\
                                   objective = 'binary:logistic',\
                                   early_stopping_rounds=20,\
                                   eval_metric=["error","logloss"],\
                                   )

eval_set = [(X_train_PCA, y_train), (X_test_PCA, y_test)]
model.fit(X_train_PCA, y_train, eval_set=eval_set, verbose=False)# make predictions
predictions = model.predict(X_test_PCA)
# evaluate predictions
accuracy = accuracy_score(y_test, predictions)
precision = precision_score(y_test, predictions)

print("accuracy: %.2f%%" % (accuracy * 100.0))
print("Precision: %.2f%%" % (precision * 100.0))
print(classification_report(y_test, predictions, labels=[1,0]))
```

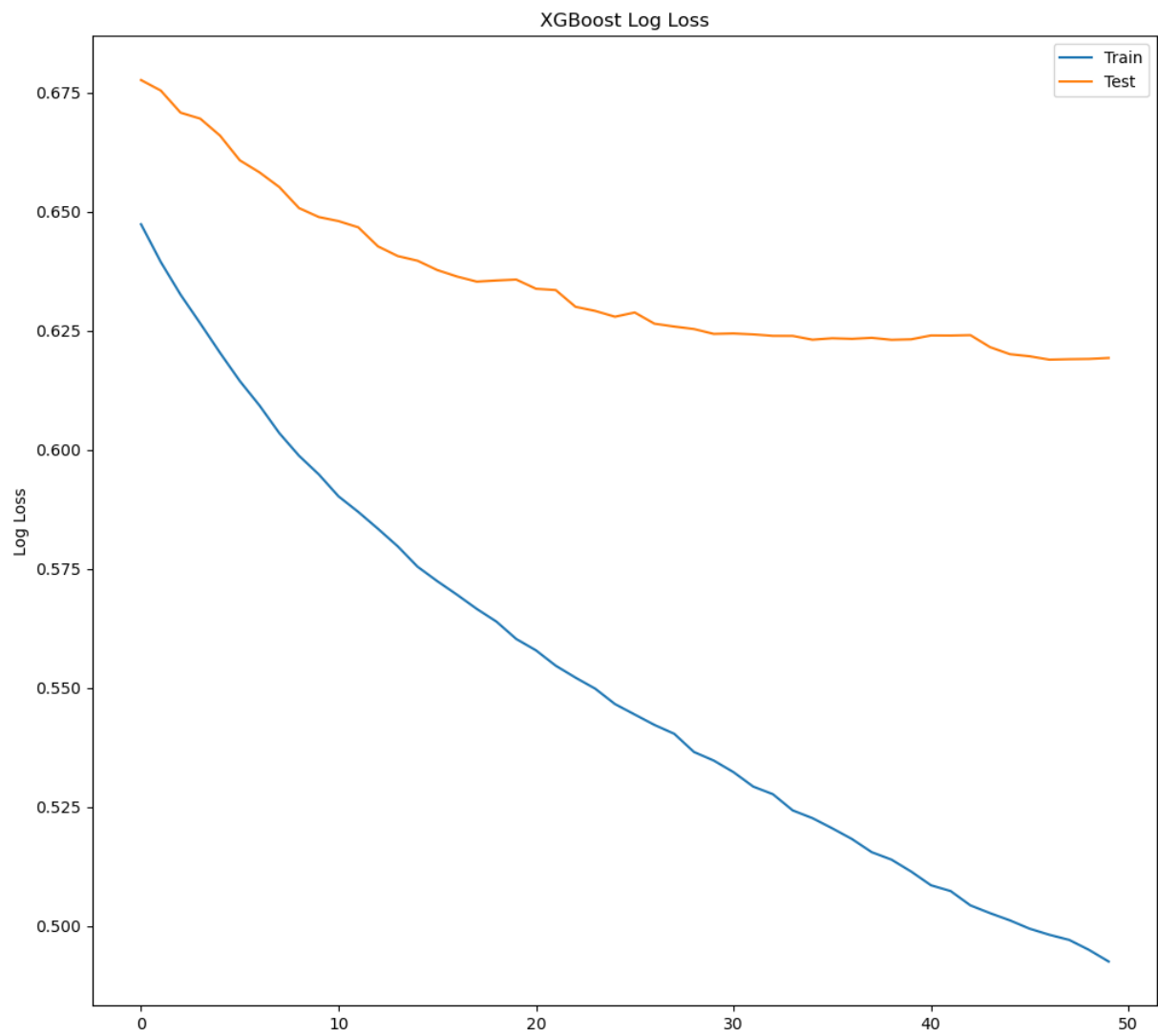
accuracy: 66.37%

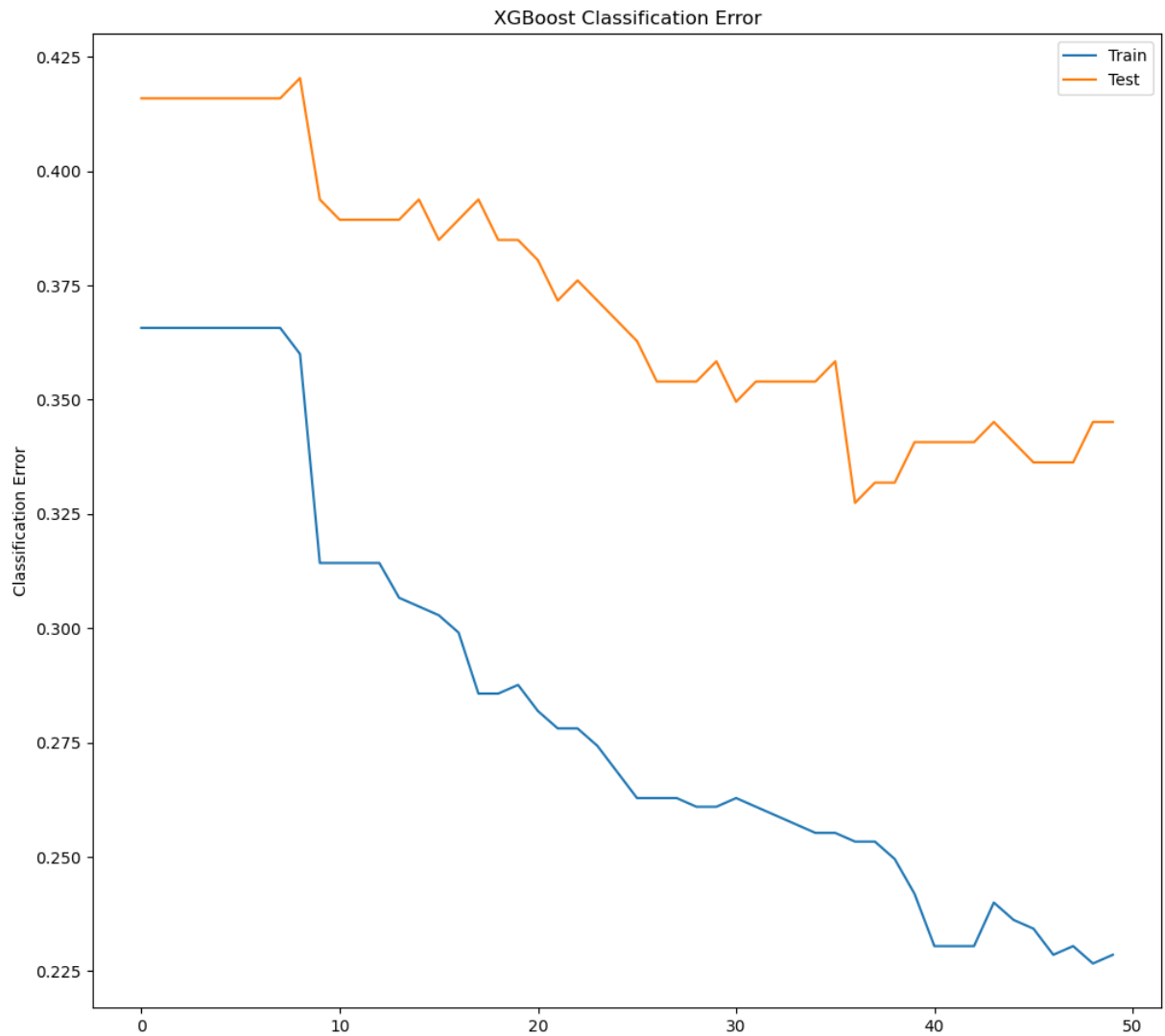
Precision: 68.00%

	precision	recall	f1-score	support
1	0.68	0.36	0.47	94
0	0.66	0.88	0.75	132
accuracy			0.66	226
macro avg	0.67	0.62	0.61	226
weighted avg	0.67	0.66	0.64	226

```
In [65]: # retrieve performance metrics
results = model.evals_result()
epochs = len(results['validation_0']['error'])
x_axis = range(0, epochs)
# plot log loss
fig, ax = plt.subplots()
ax.plot(x_axis, results['validation_0']['logloss'], label='Train')
ax.plot(x_axis, results['validation_1']['logloss'], label='Test')
ax.legend()
plt.ylabel('Log Loss')
plt.title('XGBoost Log Loss')
plt.show()
# plot classification error
fig, ax = plt.subplots()
ax.plot(x_axis, results['validation_0']['error'], label='Train')
ax.plot(x_axis, results['validation_1']['error'], label='Test')
ax.legend()
plt.ylabel('Classification Error')
```

```
plt.title('XGBoost Classification Error')  
plt.show()
```





```
In [66]: # confusion matrix for the test data
cm = sk.metrics.confusion_matrix(y_test, predictions, labels=[1,0])

fig, ax = plt.subplots(figsize=(10,10))
sns.heatmap(cm, annot=True, fmt='g', ax = ax);

# labels, title and ticks
ax.set_xlabel('Predicted labels');
ax.set_ylabel('True labels');
ax.set_title('Confusion Matrix');
ax.xaxis.set_ticklabels(['Sensible', 'Not sensible']);
ax.yaxis.set_ticklabels(['Sensible', 'Not sensible']);
```

